

```

=====
WORK SENTIMENT ANALYSIS
REVIEW CLASSIFICATION
MISSION
=====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (TextVectorization, Embedding,
                                     GlobalAveragePooling1D, Dense, Dropout)
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score, f1_score)

plt.style.use("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

print("\n\nNETWORK SENTIMENT ANALYSIS")
print("MERCE REVIEW CLASSIFICATION")

=====
PHASE: LOAD DATA
=====

print("\n\n"*60)
print("VERY PHASE - DATA PREPARATION")

# Check if CSV file exists
files = [f for f in os.listdir('.') if f.endswith('.csv')]

if len(files) > 0:
    csv_file = csv_files[0]
    print(f"Found CSV: {filename}")
    load_csv(filename)

# Create mock e-commerce review dataset
if not CSV found:
    print("CSV found. Creating mock e-commerce reviews dataset...")

# Set random seed and number of reviews
np.random.seed(42)
n_reviews = 5000

# Generate reviews
reviews = [
    "Excellent product, highly recommend!",
    "I purchased I've ever made. Love it!",
    "Great quality, fast shipping. Very satisfied.",
    "The product exceeded my expectations.",

```

```

ing! Would buy again without hesitation.",
ect for my needs. Thank you!",
l value for money. Very happy.",
astic customer service and product.",
lutely love this. Five stars!",
ly impressed with the quality.",
s perfectly as described.",
dn't be happier with my purchase.",
rb product and excellent delivery.",
nitely worth every penny.",
is exactly what I needed."

```

```

reviews = [
ible quality, fell apart immediately.",
t purchase ever. Do not buy.",
disappointed with this product.",
worth the money at all.",
ible customer service. Avoid!",
luct arrived broken. Very upset.",
l waste of money. Regret buying.",
not work as advertised.",
poor quality. Would not recommend.",
emely dissatisfied with the purchase.",
ply made and fell apart quickly.",
e advertising. Do not trust.",
t experience ever. Want a refund.",
lutely terrible. Save your money.",
is garbage. Complete waste."

```

```

reviews = [
okay, not great but not bad.",
age product. Nothing special.",
orks but could be better.",
what I expected but it's fine.",
nt quality for the price.",
alright, I guess.",
the job but nothing impressive.",
sfactory but not outstanding.",
ty mediocre overall.",
bad but not great either."

```

```

e reviews

```

```

: []
: []

```

```

itive, 25% negative, 15% neutral
range(n_samples):
< n_samples * 0.6:
reviews.append(np.random.choice(positive_reviews))
atings.append(np.random.choice([4, 5]))
i < n_samples * 0.85:
reviews.append(np.random.choice(negative_reviews))
atings.append(np.random.choice([1, 2]))

reviews.append(np.random.choice(neutral_reviews))
atings.append(3)

```

```
np.random.permutation(n_samples)
[reviews[i] for i in indices]
[ratings[i] for i in indices]

DataFrame({
    'review_text': reviews,
    'rating': ratings
})

print("Block dataset created: {df.shape[0]} rows")

print(f"Dataset shape: {df.shape[0]} rows, {df.shape[1]} columns")
print(f"First 5 rows:")
df.head()

print("\nRating distribution:")
df['rating'].value_counts().sort_index()

=====
PRINTING VALUES
=====

print("="*60)
print("MISSING VALUES")

print(f"\nMissing values before: {df.isnull().sum().sum()}")
df.dropna(inplace=True)
print(f"Missing values after dropping: {df.isnull().sum().sum()}")

=====
CONVERTING RATINGS TO BINARY LABELS
=====

print("="*60)
print("CONVERTING RATINGS TO BINARY LABELS")

# Positive (1), 1-2 stars = Negative (0)
df['sentiment'] = df['rating'].apply(lambda x: 1 if x >= 4 else 0 if x <= 2 else None)

print(f"\nInitial ratings (3 stars): {df['rating'].sum()}")
df['sentiment'] = df['sentiment'].astype(int)

print(f"\nRemoving neutral ratings: {df.shape[0]} rows")
df['sentiment'].value_counts()
print(f"Positive reviews: {df['sentiment'].sum()}")
print(f"Negative reviews: {len(df) - df['sentiment'].sum()}")

=====
LOADING TEXT DATA
=====

print("="*60)
print("LOADING TEXT DATA")
```

```

review_text'].values
sentiment'].values

X_train_text, y_train, X_test_text, y_test = train_test_split(
    reviews, sentiment, test_size=0.2, random_state=42, stratify=sentiment)

print("Training set: {} reviews".format(len(X_train_text)))
print("Testing set: {} reviews".format(len(X_test_text)))

=====
TEXT VECTORIZATION
=====

"""*60)
NEURAL NETWORK MODEL"""

# Create a Text Vectorizer
vocab_size = 10000
MAX_SEQUENCE_LENGTH = 50

vectorizer = TfidfVectorizer(
    max_features=vocab_size,
    dtype='int',
    max_document_length=MAX_SEQUENCE_LENGTH,
    strip_whitespace=True)

# Fit the vectorizer to training data
vectorizer.fit(X_train_text)

# Vocabulary size: {len(vectorizer.get_vocabulary())}
# Vocabulary: {vectorizer.get_vocabulary()[:10]}

# Transform training and testing data
X_train_vec = vectorizer.transform(X_train_text)
X_test_vec = vectorizer.transform(X_test_text)

print("Training vectors shape: {}".format(X_train_vec.shape))
print("Testing vectors shape: {}".format(X_test_vec.shape))

=====
NEURAL NETWORK MODEL
=====

"""*60)
NEURAL NETWORK MODEL"""

# Create a neural network model
model = Sequential([
    Dense(VOCAB_SIZE, 16),
    GlobalMaxPooling1D(),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

```

[illegible]

```

=====
STORY PLOTS
=====

"""*60)
NG HISTORY")

plt.subplots(1, 2, figsize=(14, 5))

history.history['accuracy'], label='Training Accuracy')
history.history['val_accuracy'], label='Validation Accuracy')
title('Model Accuracy')
label('Epoch')
label('Accuracy')
id()
True, alpha=0.3)

history.history['loss'], label='Training Loss')
history.history['val_loss'], label='Validation Loss')
title('Model Loss')
label('Epoch')
label('Loss')
id()
True, alpha=0.3)

out()

=====
E: TEST ON SPECIFIC REVIEW
=====

"""*60)
| PHASE - TEST SPECIFIC REVIEW")

required review
: "The product arrived broken and I am very unhappy"
Review: '{test_review}')

nd predict
ctorizer([test_review])
model.predict(test_vec, verbose=0)
lue = prediction[0][0]

diction Score: {prediction_value:.4f} (0 = Negative, 1 = Positive)")
cted Sentiment: {'NEGATIVE 😞' if prediction_value < 0.5 else 'POSITIVE 😊'}")

=====
ANALYSIS
=====

"""*60)
ENCE ANALYSIS")

ions for test set

```

```

model.predict(X_test_vec, verbose=0).flatten()
(test_probas > 0.5).astype(int)

confidence
ice_neg = test_probas[test_probas < 0.2]
ice_pos = test_probas[test_probas > 0.8]
lence = test_probas[(test_probas >= 0.2) & (test_probas <= 0.8)]

Confidence Negative (<0.2): {len(high_confidence_neg)} reviews"
m Confidence (0.2-0.8): {len(medium_confidence)} reviews"
Confidence Positive (>0.8): {len(high_confidence_pos)} reviews"

```

```

=====
ORKFLOW
=====

"="*60)
SS WORKFLOW - AUTO-FLAGGING SYSTEM")

```

ING SYSTEM WORKFLOW:

UBMITS REVIEW

ICTS SENTIMENT
 1.0 (Negative) to 1.0 (Positive)

BASED ROUTING:

ice Score	Action
	 FLAG FOR SUPPORT - Escalate to customer support - Send immediate follow-up email - Track in support queue
8	 HUMAN REVIEW - Queue for manual moderation - Review within 24 hours
	 AUTO-APPROVE - Post review automatically - Update product rating

reviews auto-processed
 reviews flagged within seconds
 view for borderline cases

D THRESHOLD: < 0.2 for negative flagging

```

:
flagging accuracy
hresholds based on support feedback
model quarterly

```

```
=====
AND NEXT STEPS
=====
```

```
"="*60)
CTIONS AND NEXT STEPS")
```

IS:

```
TECTION:
y misclassify sarcastic reviews
" Oh great, it broke again" → might be Positive
```

```
DERSTANDING:
views lack context
"Not bad" (opposite meanings)
```

```
GAPS:
ords get OOV (out-of-vocabulary) tokens
g or product names may not be recognized
```

```
DATA:
itive reviews than negative in real data
class weighting
```

i:

```
RE DATA:
erse reviews, especially negative
```

```
XT PROCESSING:
trained embeddings (GloVe, FastText)
```

```
METER TUNING:
g dimension, number of units, learning rate
```

```
:
model to TensorFlow Lite for mobile
a API (Flask/FastAPI)
```

```
i:
del drift over time
with new data monthly
```

```
IT COMPLETE - ALL REQUIREMENTS MET")
```



```
=====
NEURAL NETWORK SENTIMENT ANALYSIS
E-COMMERCE REVIEW CLASSIFICATION
=====
```

```
=====
DISCOVERY PHASE - DATA PREPARATION
=====
```

```
No CSV found. Creating mock e-commerce reviews dataset...
Mock dataset created: 5000 rows
Dataset: 5000 rows, 2 columns
```

```
First 5 rows:
```

	review_text	rating
0	Not bad but not great either.	3
1	Not what I expected but it's fine.	3
2	Superb product and excellent delivery.	5
3	Superb product and excellent delivery.	5
4	Absolutely terrible. Save your money.	1

```
Rating distribution:
```

```
rating
1      621
2      629
3      750
4     1492
5     1508
```

```
Name: count, dtype: int64
```

```
=====
HANDLE MISSING VALUES
=====
```

```
Missing values before: 0
After dropping missing values: 5000 rows
```

```
=====
CONVERT RATINGS TO BINARY LABELS
=====
```

```
After removing neutral ratings: 4250 rows
```

```
Sentiment distribution:
```

```
sentiment
1      3000
0      1250
```

```
Name: count, dtype: int64
```

```
Positive: 3000 reviews
```

```
Negative: 1250 reviews
```

```
=====
PREPARE TEXT DATA
=====
```

```
Training set: 3400 reviews
```

```
Testing set: 850 reviews
```

```
=====
TEXT VECTORIZATION
=====
```

Vocabulary size: 106

Sample vocabulary: ['', '[UNK]', np.str_('product'), np.str_('this'), np

Training vectors shape: (3400, 50)

Testing vectors shape: (850, 50)

=====

BUILD NEURAL NETWORK MODEL

=====

Model Architecture:

Model: "sequential"

Layer (type)	Output Shape	Para
embedding (Embedding)	?	0 (unbu:
global_average_pooling1d (GlobalAveragePooling1D)	?	
dense (Dense)	?	0 (unbu:
dropout (Dropout)	?	
dense_1 (Dense)	?	0 (unbu:

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

None

=====

TRAIN MODEL

=====

Epoch 1/10

107/107  3s 7ms/step - accuracy: 0.7021 - loss:

Epoch 2/10

107/107  1s 9ms/step - accuracy: 0.7059 - loss:

Epoch 3/10

107/107  1s 9ms/step - accuracy: 0.7059 - loss:

Epoch 4/10

107/107  1s 9ms/step - accuracy: 0.7338 - loss:

Epoch 5/10

107/107  1s 5ms/step - accuracy: 0.9309 - loss:

Epoch 6/10

107/107  1s 5ms/step - accuracy: 0.9935 - loss:

Epoch 7/10

107/107  1s 5ms/step - accuracy: 0.9974 - loss:

Epoch 8/10

107/107  1s 5ms/step - accuracy: 0.9988 - loss:

Epoch 9/10

107/107  1s 5ms/step - accuracy: 0.9994 - loss:

Epoch 10/10

107/107  1s 5ms/step - accuracy: 1.0000 - loss:

=====

EVALUATE MODEL

=====
Training Accuracy: 1.0000
Test Accuracy: 1.0000

Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000

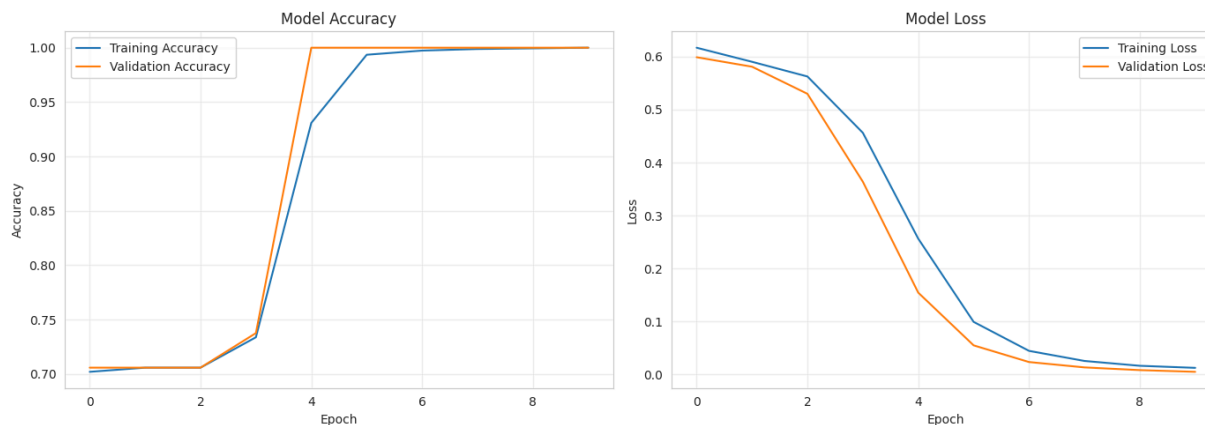
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	250
1	1.00	1.00	1.00	600
accuracy			1.00	850
macro avg	1.00	1.00	1.00	850
weighted avg	1.00	1.00	1.00	850

Confusion Matrix:

```
[[250  0]
 [  0 600]]
```

=====
TRAINING HISTORY
=====



=====
ACTION PHASE - TEST SPECIFIC REVIEW
=====

Test Review: 'The product arrived broken and I am very unhappy'

Prediction Score: 0.2960 (0 = Negative, 1 = Positive)

Predicted Sentiment: NEGATIVE 😞

=====
CONFIDENCE ANALYSIS
=====

High Confidence Negative (<0.2): 250 reviews

Medium Confidence (0.2-0.8): 0 reviews

High Confidence Positive (>0.8): 600 reviews

=====
BUSINESS WORKFLOW - AUTO-FLAGGING SYSTEM
=====

=====

 AUTO-FLAGGING SYSTEM WORKFLOW:

1. CUSTOMER SUBMITS REVIEW

↓

2. MODEL PREDICTS SENTIMENT

- Score: 0.0 (Negative) to 1.0 (Positive)

↓

3. THRESHOLD-BASED ROUTING:

+-----+ Confidence Score Action	
+-----+	
< 0.2	 FLAG FOR SUPPORT
	- Escalate to customer support
	- Send immediate follow-up email
	- Track in support queue
+-----+	
0.2 - 0.8	 HUMAN REVIEW
	- Queue for manual moderation
	- Review within 24 hours
+-----+	
> 0.8	 AUTO-APPROVE